

Classification Analysis of Portuguese Bank Marketing Data: Targeting Term Deposit Subscribers

1. Understand the Problem & Data

The dataset was obtained from the UC Irvine Machine Learning Repository, and it dives into a Portuguese bank's marketing campaign, where customers are contacted and offered a term deposit. For each call and client, the database has demographics, financial information, client details, and an outcome represented by a variable:

- 'y' – whether the client subscribed to the term deposit ('yes' or 'no').

The bank focuses on customers who are more likely to subscribe to a term deposit, so they can target leads they find promising. This outcome is the most beneficial and important aspect of the prediction, because identifying likely subscribers increases revenue and returns on what was spent on the marketing campaign. The misclassification of a subscriber can lead to loss in term deposit, emphasizing the importance of correct classification.

For this reason, the chosen important class was $y = \text{'yes'}$ (client does subscribe to the term deposit). This analysis depicts how well models identify the 'yes' class.

The full dataset contains 45,211 observations and 17 variables. The variables are as follows:

Dependent Variable:

- **"y"** : A binary data type that portrays if the client is subscribed (**yes**) or not (**no**).

Quantitative Independent Variables:

- **age**: Age
- **balance**: Average yearly balance in euros
- **campaign**: Number of contacts in current campaign
- **pdays**: Days since prior contact (-1 = never previously contacted)
- **previous**: Number of contacts before this campaign

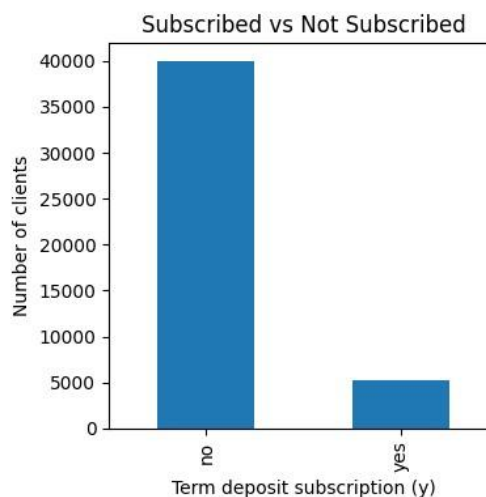
Other independent variables, both categorical and binary, which are included in the dataset, but I will not be using, because the requirements of the project ask you to only consider quantitative predictor variables, are: job (type of job), marital (marital status), education (education level), default (credit in default?), housing (has housing loan?),

loan (has personal loan?), contact (communication type), day (last contact day), month (last contact month), and poutcome (outcome of last campaign).

Regarding limitations, the variable “duration” is only known after a call ends, and although it is highly predictive of “y”, it should only be used for benchmarking and/or discarded if seeking a realistic predictive model. I will exclude this variable from the predictive models because it would not be available before the marketing call is completed.

A high class imbalance is present, as we will see in our following plot. Moreover, campaign effectiveness could vary on a monthly basis, or by economic context, consideration should be applied during cross-validation and splitting data when needed.

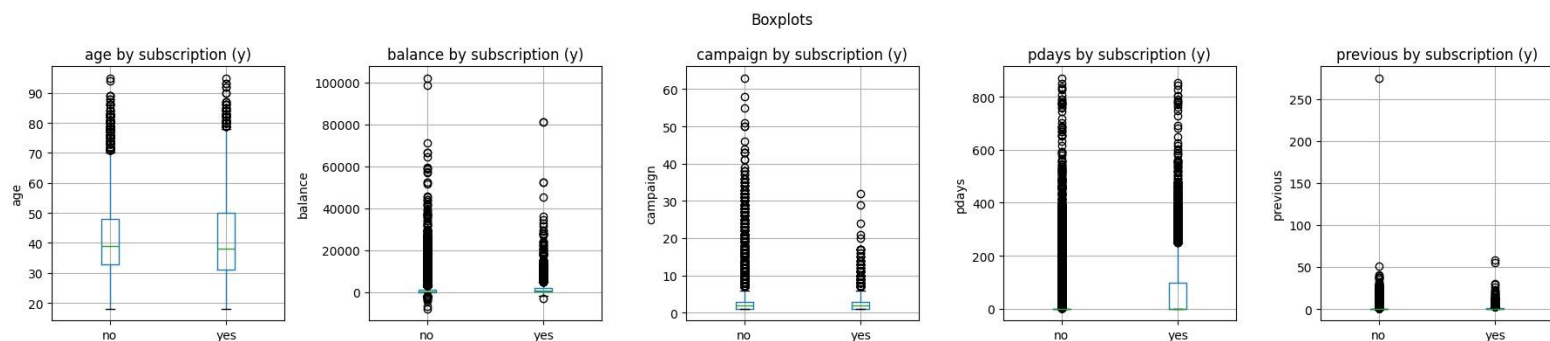
Observing the value counts, we see that 39,922 (88.3%) are not subscribed, while 5,289 (11.7%) are subscribed.



The bar chart portrays the imbalance in the dataset, where non-subscribers are the large majority. This observation will prove important in our assessment, where a model can achieve high overall accuracy by predicting ‘no’ for most of the customers, but still portraying poor performance on the important ‘yes’ class.

Summarizing the predictors one finds that age ranges from 18 to 95, with mean centered at 41. Balance has a large spread, and a large standard deviation, portraying a heavy skew and many outliers present. Campaign is usually small, with a median of 2 but maximum of 63. Pdays is mostly -1, meaning never previously contacted, but can go as high as 871 for a couple of clients. Previous is mostly 0, and few clients have been contacted many times before.

To understand how the chosen variables relate to the term deposit subscription, I visualized boxplots of each quantitative predictor by y:



For age by subscription, the median age of subscribers is slightly higher than nonsubscribers. Older clients are somewhat more likely to subscribe to term deposits compared to younger clients.

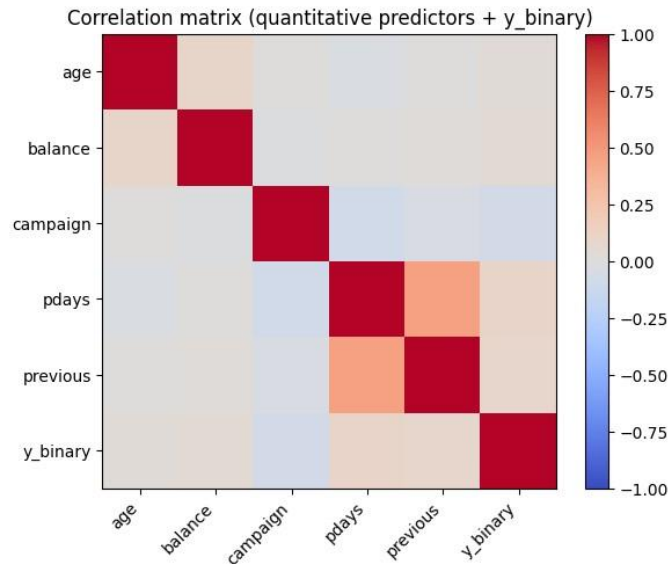
For balance by subscription, higher account balances are present in subscribers, with higher median compared to non-subscribers. The distribution appears skewed, so we visualize a few extreme balances that affect the spread.

For campaign by subscription, the distribution of campaign is shifted to fewer contacts. Many subscribed clients were reached with less calls. Non-subscribers, however, tend to have more repeated contacts.

For pdays by subscription, the boxplots show a large difference, where subscriptions occur when pdays is not in the low end. However, pdays should be interpreted carefully because -1 represents customers who were never previously contacted, so the variable reflects both prior-contact status and timing rather than only recency.

For previous by subscription, there is a tendency for subscribers to have higher values of previous on average, meaning that subscribers in this dataset had more prior campaign contacts compared to non-subscribers. Nevertheless, for both cases, the distribution is concentrated near zero.

I also created a correlation matrix heatmap to better visualize the relationship between variables:



Correlations among the chosen variables are not high, although pdays and previous do show a positive correlation. This relationship should be interpreted carefully because pdays also uses -1 to represent customers who were never previously contacted, so it reflects prior-contact status as well as elapsed time.

The correlations with y_binary are relatively small, the largest being for pdays, which is positive, previous also positive, balance also positive, and campaign which portrays a small negative correlation.

These signs match the patterns in the boxplots, where more subscriptions are associated with higher balances, differences in prior-contact history captured by pdays, and slightly more previous contacts, while fewer subscriptions are associated with repeated contact within the current campaign. No single quantitative variable is completely definitive in deciding subscriber vs non-subscriber, but can aid the direction useful for our classification.

2. KNN Model Specification

The k-nearest neighbors (KNN) classifier is distance based. To predict the outcome of a new customer, the algorithm:

- Computes distance between new customer and every customer in the training set in the predictor space.
- Identifies k closest observations – hence the neighbors.
- Assigns class label to the new customer by majority among the k neighbors

The binary response ($y = 1$ if subscribes, $y = 0$ otherwise) means that the model predicts the probability of subscription as the proportion of 'yes' among k nearest neighbors, and then classifies a customer as 'yes' if the proportion goes over 0.5. I use Euclidean distance as the distance metric.

To keep consistency and alignment with the other models I will build, we use the same five predictor variables: age, balance, campaign, pdays and previous. We intentionally restrict the model to these quantitative variables because KNN performance can go wrong in many-dimensions spaces, like when adding many dummy variables. Using the same predictors as we will for the tree-based models will give us a clean comparison across methods and help us conclude on the effects of different models on performance.

The specification choices are:

- The distance metric, Euclidean distance in the 5-dimensional predictor space.
- Number of neighbors, k, which controls the decision boundary.

KNN relies on distances, meaning that the scale of the predictors do matter. With predictors on original scales, variables with very large ranges can dominate the Euclidean distance calculation, with small scale variables left behind.

I normalize quantitative predictors using min-max scaling, which can be expressed in the following formula:

$$\frac{x - \text{mintrain} * (x)}{\text{maxtrain} * (x) - \text{mintrain} * (x)}$$

During model selection, min-max scaling is included inside a pipeline so that the minimum and maximum for each predictor are learned separately within each cross-validation training fold. After selecting the final value of k, the pipeline is fit on the full training set and then applied to the holdout test set. This keeps the test data separate from both preprocessing and model selection.

Normalizing data impacts KNN because it ensures all five predictors contribute on a comparable scale. It can vary what is considered a nearest neighbor, especially with the large spread we saw before. Normalization can change which observations are considered nearest neighbors and may affect predictive performance compared with using predictors on their original scales.

3. KNN Model Estimation

Using the predictors in their original units, a sequence of KNN classifiers with different k's was fit. For each value of k from 1 to 30, I trained the model using the training set and calculated 5-fold cross-validation accuracy. The value of k was selected using cross-validation performance on the training data only, keeping the holdout test set separate until after model selection.

This search's best KNN model among the tested values with unscaled predictors used:

- $k = 30$
- Training accuracy = 0.884
- Cross-validation accuracy = 0.883
- Test accuracy = 0.884

Smaller values of k showed much higher training accuracy but lower cross-validation accuracy, depicting overfitting. As k increased, training accuracy decreased while cross-validation performance became more stable. Among the values tested from 1 to 30, $k = 30$ achieved the highest cross-validation accuracy. I therefore selected $k = 30$ before evaluating the model on the holdout test set.

K	TRAIN ACCURACY	CV ACCURACY
26	0.884	0.883
27	0.884	0.882
28	0.884	0.883
29	0.884	0.882
30	0.884	0.883

I repeated the procedure but utilized min-max normalization on the predictors, as was specified in the Model Specification. The scaling step was included inside the model pipeline, allowing the minimum and maximum values to be learned separately within each cross-validation training fold. After selecting k , the final pipeline was fit on the full training set and evaluated on the holdout test set.

With normalized predictors, KNN models for $k = 1, 2 \dots 30$ were fit. The best performing model based on cross-validation accuracy had:

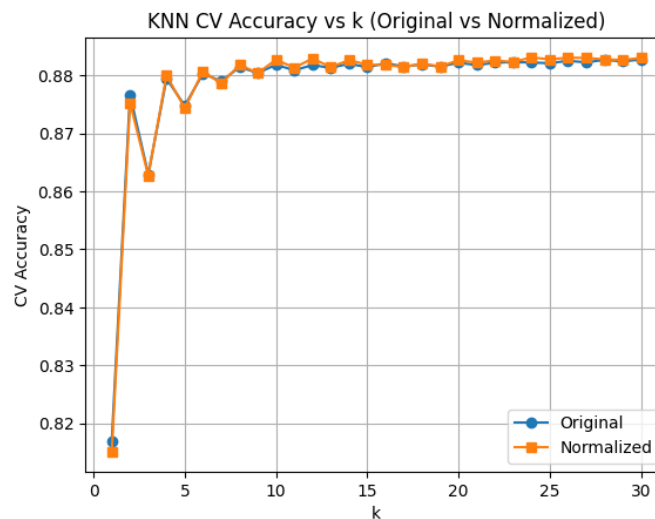
- $k = 24$
- Training accuracy = 0.887
- Cross-validation accuracy = 0.883
- Test accuracy = 0.883

Again, very small k 's led to high training accuracy but lower cross-validation performance. As k increased, the difference between training and cross-validation accuracy narrowed. $k = 24$ provided the highest cross-validation accuracy among the tested normalized models, so I adopt $k = 24$ as the normalized KNN specification before evaluating it on the holdout test set.

K	TRAIN ACCURACY	CV ACCURACY
22	0.887	0.883
23	0.887	0.882

24	0.887	0.883
25	0.886	0.883
26	0.887	0.883

Normalization impacts the analysis, and comparing the two versions we see some differences. Normalizing the predictors changed the selected k from 30 to 24 and produced slightly higher cross-validation accuracy, although the final test accuracies of the two models were nearly identical.



Variables such as pdays and balance have much larger numbers than campaign and previous. Without normalization these variables can dominate the Euclidean distance calculation and weigh down the importance of smaller-scale predictors. With normalization, each predictor contributes on a more comparable scale to the distance calculation.

In this dataset, normalization does not transform the KNN model into a drastically better classifier. The unscaled and normalized models achieved nearly identical test accuracy, while the normalized model achieved slightly higher cross-validation accuracy and provides a more appropriate distance calculation because the predictors are placed on comparable scales. For these reasons, I will rely on the normalized KNN model where $k = 24$ for comparison to the other models later in my report.

4. Decision Tree Model Specification

For the decision tree model, I utilize a classification tree using `DecisionTreeClassifier`. The response is in the following binary form: $y = 1$ if the client subscribes, $y = 0$ if otherwise. The tree recursively partitions the space of predictors in regions making

binary splits. At each split, the predictor and threshold which make the best separation in between classes based on a measure of impurity are chosen.

The splitting criterion used is Gini impurity, where Gini impurity at a node with class probabilities of p_0 and p_1 are:

$$P(Y = 0) \cdot (1 - P(Y = 0)) + P(Y = 1) \cdot (1 - P(Y = 1))$$

simplified to:

$$G = 1 - p_0^2 - p_1^2$$

The split with largest reduction in impurity is chosen. The final prediction for each observation is the majority class of the terminal node it ends in.

As in the KNN Model, the decision tree is restricted to predictors: age, balance, campaign, pdays, and previous. These variables are available before campaign outcome known, quantitative, and the same set I used for KNN and will use for my other models, to keep the comparisons consistent.

Again, duration is a variable excluded, even though highly predictive, its only observed post call has taken place, and wouldn't be available for prediction of new customer. Categorical variables are not included in this project, we focus on key quantitative predictors.

For a decision tree, the key choices for modeling are:

- Splitting:
 - Gini impurity: default for DecisionTreeClassifier, standard and wellperforming, we do not treat this as a tuning parameter.
- Complexity:
 - We tune max_depth, or the maximum depth of a tree, the number of splits from the root to the leaf.
 - A small max_depth is a simple tree with higher bias but lower variance.
 - A large max_depth is a complex tree with lower bias but higher variance and overall more risk of overfitting.
 - I choose max_depth using the same 80% training and 20% testing split used for KNN, stratified to preserve the subscriber/non-subscriber class balance. On the training set, I evaluate depths 1-10 using 5-fold stratified cross-validation, recording mean CV accuracy and training accuracy for each depth. I select the depth with the highest cross-validation accuracy.
- Reproducibility:
 - Set random_state = 0 to keep results reproducible and consistent.

5. Decision Tree Model Estimation

To estimate our decision tree, I utilized the validation set approach as I did for KNN. The predictors and response were defined:

- Predictors: age, balance, campaign, pdays, previous
- Response: $y_{\text{binary}} = 1$ if $y = \text{'yes'}$. Otherwise, $y_{\text{binary}} = 0$.

The data is split 80% training and 20% testing, stratified by the response variable, with `random_state = 0` for reproducibility. On the training set, the main parameter `max_depth` is tuned by fitting trees with depths 1-10 and measuring 5-fold cross-validation accuracy and training accuracy at each depth.

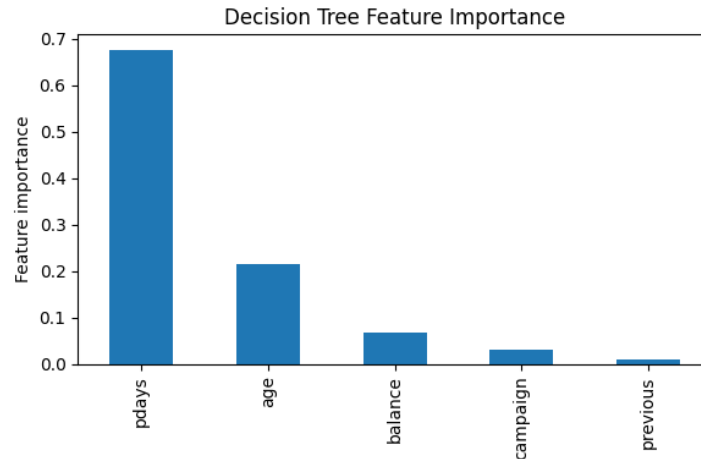
MAX_DEPTH	CV_ACCURACY	TRAIN_ACCURACY
2	0.883	0.883
3	0.884	0.885
4	0.885	0.886
5	0.885	0.886
6	0.885	0.887

The results portray CV accuracies ranging from approximately 0.881 to 0.885. Training accuracy increases steadily as depth increases, while cross-validation accuracy peaks at `max_depth = 6`. At this depth, CV accuracy is approximately 0.885 and training accuracy is approximately 0.887. Beyond depth 6, training accuracy continues to increase while cross-validation accuracy declines, suggesting that additional complexity begins to overfit the training data rather than improve out-of-sample performance.

The chosen complexity was `max_depth = 6`, after which I refit the tree on the full training set and evaluated it on the holdout test set:

Train Accuracy	0.887
Test Accuracy	0.883

The chosen `max_depth` gives the highest cross-validation accuracy among the depths tested, while maintaining a relatively small train-to-test gap and a tree that remains interpretable.



As a complement to the tree, we can observe the feature importances. They are concentrated in $\text{pdays} \approx 0.675$, $\text{age} \approx 0.216$, $\text{balance} \approx 0.068$, $\text{campaign} \approx 0.031$, and $\text{previous} \approx 0.011$. This supports the conclusion that pdays dominates the fitted tree, followed by age , while balance , campaign , and previous contribute smaller amounts to the model's splits.

6. Bagged Tree Model Specification

Bagging is an ensemble method aiming to reduce variance of a base learner like a decision tree. For my analysis, the base learners are classification trees using Gini impurity and binary splits, with the same predictor set used in the previous section, following a procedure:

- Drawing many bootstrap samples from train data.
- For each sample, fitting a full classification tree using the same predictors and response as we did for a single-tree model.
- To predict a new observation, its passed down each tree in the ensemble and the trees decide. For a classification, the final prediction is the majority class among the predictions made by the trees.

Each tree is fit on a different bootstrap version of the data, the trees depict diversity. The majority vote over many trees reduces the variance in the model compared to a single decision tree, but still keeping the bias at the same time. Bagging can improve out-of-sample prediction compared with a single tree by averaging across many high-variance trees.

I implement bagging utilizing `RandomForestClassifier` with the choice that all predictors are considered at each split. The model behaves as a pure bagged tree ensemble rather than a random forest.

To keep the bagged tree comparable to the single tree and KNN models, we utilize the same quantitative predictors: age , balance , campaign , pdays , and previous . These

variables are numeric, available before the campaign outcome is known. Using the same predictors as we did for the other models allows performance differences to be attributed to the specific modeling technique, not differences in information.

With each bootstrap sample, the individual trees are fit using the same five predictors, but unlike the selected single decision tree, the individual ensemble trees are allowed to grow deeply. Aggregating predictions across the trees helps reduce the variance of any one individual tree.

The main parameter relates to the number of trees in the ensemble:

- `n_estimators`: The numbers of trees. Few trees lead to unstable ensemble, with varying performance depending on the bootstrap samples drawn. Increasing the number of trees, model prediction stabilizes and validation performance either improves or reaches a stopping point.

For bagging with `RandomForestClassifier`:

- `max_features = p`, where `p = 5`, or the number of predictors. ○ All predictors are considered at each split.
- `bootstrap = True` to ensure sampling with replacement.
- `random_state = 0` for reproducibility.
- Trees are deep, each individual tree is a low bias and high variance model that will benefit from averaging, or the majority vote.

To choose the number of trees, we compare:

$$n_estimators \in \{10, 100, 1000\}$$

I utilize 5-fold cross-validation for the candidate values on the training set to estimate out-of-sample accuracy. The chosen `n_estimators` is the value with the highest cross-validation accuracy among the tested candidates. I also compare whether increasing the number of trees produces meaningful additional improvement.

With this tuning strategy, the bagged tree model remains consistent and reasonable, providing a low variance way of comparing against the previous and remaining models I will go over.

7. Bagged Tree Model Estimation

To estimate the bagged tree model, we use `RandomForestClassifier` with `max_features = 5`, equal to the number of predictors, so they are all considered at each split. This reproduces a pure bagged tree ensemble. The response is `y = 1` for subscribers and `y = 0` for non-subscribers, with predictors `age`, `balance`, `campaign`, `pdays`, and `previous`.

Using the training set, we tune the primary parameter `n_estimators`, evaluating 10, 100, and 1000 trees. For each value of `n_estimators`, a bagged tree model is fit and evaluated using 5-fold cross-validation accuracy.

N_ESTIMATORS	CV ACCURACY
10	0.866
100	0.868
1000	0.868

The highest exact cross-validation performance occurred when `n_estimators` was equal to 100, with a CV accuracy of approximately 0.868. Although the 100-tree and 1000-tree models both round to 0.868 at three decimal places, the 100-tree model had the slightly higher exact CV accuracy. I then fit the bagged tree with 100 trees on the full training set and evaluated it on the holdout test set, where I obtained the following accuracies.

Training Accuracy	0.992
Test Accuracy	0.868

The training accuracy is very high, nearing perfection, while the cross-validation and test accuracies are both approximately 0.868. This shows a large gap between training and out-of-sample performance, although the similarity between cross-validation and test accuracy suggests that the holdout result is consistent with what was observed during model selection. Increasing `n_estimators` from 10 to 100 produces a small improvement in CV accuracy, while increasing the ensemble further to 1000 trees does not provide additional improvement.

I select 100 trees as the final bagged tree specification because it achieves the highest exact cross-validation accuracy among the tested candidates. Increasing the ensemble to 1000 trees does not provide additional validation improvement, while the final test accuracy remains consistent with the cross-validation result.

8. Random Forest Model Specification

Random forests extend bagged trees, reducing variance further by adding random feature selection at each split. The algorithm does the following:

- Draws bootstrap samples from the train data.
- Fits a classification tree on each sample, high in variance and unpruned.
- For each new observation, its passed through each tree using a majority vote over all of the trees to get the predicted class.

The difference from bagging is inside each tree, where in bagging, every split is chosen after searching all predictors, but in a random forest, at each split the algorithm selects a random subset of predictors, searching only within the subset for the best split.

The randomized feature selection reduces correlation between trees, which can make the aggregated forest prediction more stable and may improve out-of-sample performance compared with bagging. I implement this using `RandomForestClassifier`, which builds bootstrapped trees and chooses the best split among a random subset of all the available predictors, at each split.

To keep the random forest directly comparable to our previous models, I use the five quantitative predictors age, balance, campaign, pdays, and previous.

These variables are numeric and fully observed. They are available before the campaign outcome is known, and using the same predictors across model ensures that the differences are due to the specific modeling approach.

Any predictor can be chosen at a split, but differ from bagging in that each split only considers a random subset of these predictors. All five are available to the forest, but only a subset is examined at any given node.

For the random forest, the key modeling choices are:

- The number of trees (`n_estimators`), which controls how many bootstrapped trees are averaged:
 - With too few trees, predictions can be unstable; as the number of trees increases, the ensemble generally becomes more stable, with performance eventually reaching a plateau.
 - Aligning with the previous section, I consider:
$$n_estimators \in \{10, 100, 1000\}$$
 - I use 5-fold cross-validation on the training set to estimate out-of-sample performance for each value and select the `n_estimators` value that yields the highest cross-validation accuracy among the tested candidates.
- The number of predictors considered at each split, `max_features`.
 - Where $p = 5$, the total number of predictors, classification forests utilize the square root of p , so that `max_features = sqrt(p)`.
 - This enforces random feature selection and reduces the chance that a single strong variable, such as `pdays`, dominates all of the splits. This can reduce correlation among trees and may improve generalization compared with considering all predictors at every split.
- `bootstrap = True`, ensuring bootstrapping.
- `random_state = 0` for reproducibility.

This specification keeps alignment with the other models, and will serve in the comparison later on.

9. Random Forest Model Estimation

To estimate the random forest model and choose the parameter `n_estimators`, depicting the number of trees, I used `RandomForestClassifier` with predictors `age`, `balance`, `campaign`, `pdays`, `previous` and response `y = 1` if client subscribes, `y = 0` if otherwise. Set `max_features = 'sqrt'` so that each split considers a random subset of square root of 5 predictors. The bootstrap is set to `True`, for deep trees with no pruning, and `random_state` set to 0 for reproducibility.

On the training set, I compared three random forest specifications with different values, 10, 100, 1000. For each value, I fit a random forest and computed a 5-fold CV accuracy, for which I got the following:

N_ESTIMATORS	CV ACCURACY
10	0.862
100	0.860
1000	0.859

The differences appear small, but the highest cross-validated accuracy is obtained in the random forest model with 10 trees. Within the tested specifications, adding more trees beyond 10 does not improve cross-validation performance, which remains relatively stable across the candidate values.

Based on these results, I selected `n_estimators = 10` as the number of trees for my final random forest model. It achieved the highest cross-validation accuracy among the tested values, while increasing the number of trees did not provide additional validation improvement.

With the chosen `n_estimators` and `max_features` set to `'sqrt'`, I fit the model on the training and evaluated on the test set, for which I got the following:

Training Accuracy	0.975
Test Accuracy	0.857

I also built a confusion matrix on the test set, for which I got 7,613 true negatives, 372 false positives, 917 false negatives, and 141 true positives.

The high training accuracy and lower test accuracy show some overfitting, while the test accuracy remains close to the cross-validation performance used during model selection. The tuning procedure supports the 10-tree random forest as the selected

specification for this analysis. It implements random feature selection and achieves the highest cross-validation accuracy among the tested ensemble sizes.

10. Model Assessment

The following table summarizes the overall classification accuracy for the final models:

MODEL	TRAIN ACCURACY	TEST ACCURACY
KNN	0.884	0.884
NORMALIZED KNN	0.887	0.883
DECISION TREE	0.887	0.883
BAGGED TREES	0.992	0.868
RANDOM FOREST	0.975	0.857

Comparing normalized KNN and Decision Tree, we see that both have very similar train and test accuracies, where the small gaps between them suggest that neither model depicts heavy overfitting. The two models also achieve nearly identical test accuracy at approximately 0.883. The unscaled KNN has the highest test accuracy among the models at approximately 0.884, although the difference between these three models is very small.

Comparing the original KNN and the normalized KNN, min-max scaling changes the selected k from 30 to 24. The two models achieve nearly identical overall test accuracy, with the unscaled model slightly higher. However, normalization places all predictors on comparable scales for the distance calculation and, as shown by the class-specific results below, improves the model's ability to identify subscribers.

Analyzing the bagged trees, training accuracy is extremely high at 0.992, showing that the ensemble of deep trees fits the training data near perfectly. Test accuracy is lower at 0.868, showing a large train-to-test gap and some overfitting. Bagging reduces the variance of individual trees through aggregation, but the deep base trees still fit the training data much more strongly than the out-of-sample data.

Inspecting the Random Forest, it also shows a high training accuracy of 0.975 coupled with the lowest test accuracy among the models at 0.857. Like the bagged trees, this depicts some overfitting, and in this dataset the additional randomness in the splits does not translate into better overall accuracy than KNN or the single decision tree.

Analyzing overall accuracy, the unscaled KNN achieves the highest test accuracy by a very small margin, followed closely by the normalized KNN and decision tree. Nevertheless, we started this analysis stating that the dataset is highly imbalanced, with only around 12% being 'yes' in subscribing to term deposits. Overall accuracy is

dominated by the majority class and can mislead performance on the critical 'yes' class. That's why metrics specific to the important class are also analyzed.

As stated before, the important class is defined as $y = \text{'yes'}$, where the client subscribes to the term deposit. For this class the relevant metric is sensitivity, where:

$$Sensitivity_{yes} = \frac{True\ Positives}{True\ Positives + False\ Negatives}$$

Depicting the proportion of actual subscribers that the model correctly identifies. Using confusion matrices for the models, I compare the behavior on this important class, where:

$$\begin{bmatrix} TN & FP \\ FN & TP \end{bmatrix}$$

KNN, original, with $k = 30$:

$$\begin{bmatrix} 7972 & 13 \\ 1037 & 21 \end{bmatrix}$$

KNN, normalized, with $k = 24$:

$$\begin{bmatrix} 7911 & 74 \\ 980 & 78 \end{bmatrix}$$

Decision Tree, with $\text{max_depth} = 6$:

$$\begin{bmatrix} 7908 & 77 \\ 977 & 81 \end{bmatrix}$$

Bagged Trees, with 100 trees:

$$\begin{bmatrix} 7689 & 296 \\ 899 & 159 \end{bmatrix}$$

Random Forest, with 10 trees and $\text{max_features} = \text{'sqrt'}$:

$$\begin{bmatrix} 7613 & 372 \\ 917 & 141 \end{bmatrix}$$

In all cases, there are 1,058 actual 'yes' cases in the test set (FN + TP).

- KNN, original, with 21 true positives and 1,037 false negatives, has a sensitivity of approximately 2.0%.

- Normalized KNN, with 78 true positives and 980 false negatives, has a sensitivity of approximately 7.4%.
- Decision Tree, with 81 true positives and 977 false negatives, has a sensitivity of approximately 7.7%.
- Random Forest, with 141 true positives and 917 false negatives, has a sensitivity of approximately 13.3%.
- Bagged Trees, with 159 true positives and 899 false negatives, has a sensitivity of approximately 15.0%.

The original KNN model has the lowest sensitivity among all the models at approximately 2.0%, meaning that it predicts 'no' for almost all observations. This is consistent with its high overall accuracy being driven mainly by correct predictions of the majority class. Normalized KNN improves subscriber sensitivity to approximately 7.4%, showing that scaling helps the distance-based model identify more of the important 'yes' class even though overall accuracy remains nearly unchanged. The Decision Tree performs similarly, with slightly higher sensitivity of approximately 7.7%. The Random Forest reaches approximately 13.3% sensitivity, while the Bagged Trees achieve the highest sensitivity at approximately 15.0%. The bagged model correctly identifies 159 subscribers, more than any of the other models, but also generates more false positives than the KNN and decision-tree models.

The results depict the tradeoff that exists between overall accuracy and performance on the important class. The KNN models and Decision Tree obtain the highest overall test accuracies, but have low sensitivity for 'yes'. They are good at labeling 'no' but miss most subscribers. The Bagged Trees and Random Forest have lower overall accuracy but higher sensitivity for 'yes'. These models predict more 'yes' cases, meaning that they capture more subscribers, but at the tradeoff of generating more false positives and lowering overall accuracy.

From a business perspective, a false negative, or missing a customer who may subscribe to a term deposit, can cost more than making an extra call to a customer who is a non-subscriber, or a false positive. This tradeoff between subscriber capture, overall accuracy, and interpretability guides the final model decision.

11. Model Deployment

Based on the results, I select the Decision Tree with `max_depth = 6` as the model to carry forward for this baseline analysis. The Decision Tree achieves a training accuracy of 0.887 and test accuracy of 0.883. The train-test gap is small, suggesting relatively stable out-of-sample performance compared with its training performance.

The tree's sensitivity is low when looking at the important class perspective, achieving approximately 7.7%, while the Bagged Trees and Random Forest reach approximately 15.0% and 13.3%, respectively. Nevertheless, these ensemble models have lower

overall accuracy and are less directly interpretable than a single Decision Tree. My goal for this analysis was not only to focus on prediction, but also to provide insight and communicate the model in the role of a marketing manager. The tree can help explain which variables and splits are associated with higher predicted subscription likelihood and provide a usable tool for decision making.

The Decision Tree performs very similarly to the highest-accuracy models, with a test accuracy of approximately 0.883 compared with approximately 0.884 for the unscaled KNN. It also has a clear visual structure with if-then branches that are easier to interpret and communicate in a business decision setting. Combining competitive overall accuracy with interpretability, I select the Decision Tree as the primary interpretable model for this baseline analysis.

The final Decision Tree uses age, balance, campaign, pdays, and previous, our quantitative predictors. The splits and feature importances show us that:

- pdays is the most important variable.
- age comes second.
- balance, campaign, and previous contribute smaller amounts to the fitted tree.

There are patterns in the tree involving prior-contact history, age, and account balance. The first split occurs at $pdays \leq 8.5$, but this should be interpreted carefully because $pdays = -1$ represents customers who were never previously contacted. The tree also uses age, balance, campaign, and previous to further separate customers into groups with different predicted subscription outcomes. Older age groups appear in some branches predicting 'yes', but these patterns represent predictive associations rather than causal effects.

These rules are what help make informed decisions:

- Prior-contact history and timing appear to be useful dimensions for customer segmentation, but pdays should not be treated as a simple measure of recency because -1 represents customers who were never previously contacted.
- Age can provide additional segmentation within some of the tree branches, while balance, campaign, and previous offer smaller refinements.
- The tree can serve as a transparent guide for creating simple targeting segments, rather than treating individual splits as causal rules.

This model provides a transparent way to identify customer segments with different predicted subscription likelihoods when marketing resources are limited.

Across all methods, overall accuracy remains relatively high, although this must be interpreted in the context of the strong class imbalance, as only around 12% of clients subscribe to term deposits. All models still struggle to identify subscribers at the default classification threshold, and the highest sensitivity achieved is approximately 15.0%.

Taking these constraints into account, the Decision Tree offers a useful balance of competitive overall accuracy, stability, and interpretability for this baseline analysis.

If the business prioritized capturing more of the 'yes' class and was willing to sacrifice some overall accuracy and interpretability, the Bagged Trees or Random Forest would be stronger candidates. For this baseline comparison, I place greater weight on interpretability while maintaining competitive overall accuracy.

A more targeted approach using prior-contact history, timing, age, and other available customer characteristics could help the bank prioritize marketing efforts more systematically, although actual campaign efficiency and profitability would need to be validated using real campaign costs and outcomes.

Use of Generative AI Statement:

Tool: ChatGPT (GPT 5.1)

Used tool to generate code templates. Used to debug code and build nicer-looking plots. Fixed grammar mistakes and sentence cohesion.